

Swiftly parallelizing sequential array manipulations in Java with standard library methods for increased performance

Noah Disch
Faculty EMI
University of Applied Sciences
Offenburg
Offenburg, Germany
ndisch@stud.hs-offenburg.de

Abstract— While Java’s `parallelSort()` promises faster array sorting on multi-core systems, performance gains depend on array size and data type. This paper empirically evaluates the performance increase. Using the Java Microbenchmark Harness (JMH) for comparing `Arrays.sort()` against `Arrays.parallelSort()` across varying sizes of integer arrays. The empirical results demonstrate significant performance improvement with the parallel method on larger arrays but more importantly also over small array sizes due to sequential cutoffs. These cutoffs vary depending on the datatype: 4096 for primitive types due to cache efficient in place sorting and 8192 for objects due to computer architecture and memory overhead while sorting. This work demonstrates how to achieve practical parallelism gains when working with arrays.¹

Keywords—Java, sorting algorithms, performance benchmarking, parallel computing, Introduction

Parallel programs are essential for distributed systems nowadays. Splitting a sequential task into smaller subtasks and distributing them over multiple cores, leads to a speed up in execution time. “A multi-core machine will run a properly parallelized program faster than the non-parallelized version of the same program running on a single core” [1]. Java’s initial approach to parallelism relied on threads. But programming with threads will come with several challenges, e.g. the synchronization for the access of shared data or the communication between threads [2]. Neglecting those points may lead to race conditions and faulty data or even deadlocks due to incorrect placement of semaphores.

The complete source code, including the configuration and benchmark, is publicly available on GitHub [3].

A. The fork-join framework

Due to these challenges the fork-join framework was released in Java 7 [1]. This framework handles recursive divide and conquer tasks. A parallel task inherits from the `ForkJoinTask` class or a subclass of it, like `RecursiveTask` or `RecursiveAction` [4]. These classes provide a `compute()` method which on one hand executes the computation and also returns the result. To start the parallel execution of such task the `task.fork()` method must be called. To synchronize all the tasks for a total result,

the method `task.join()` must be called [4] and the result of the `compute()` method is returned. The advantage of using the fork-join framework lies in how the problems mentioned above are handled by the framework. Internally, it is all implemented with a `ThreadPool` which uses work stealing to balance the load of executing tasks among parallel threads [4]. This benefit extends to methods discussed later in this paper [1].

B. Challenges of the fork-join framework

But the fork-join framework does not solve all the problems mentioned in the introduction. There are still some obstacles to overcome. [4] describes several so called “anti-patterns” when using the fork-join framework, which will lead to decreased performance. One of these anti-patterns is unnecessary forking. Consider splitting an array into two subarrays to sort in the Mergesort algorithm. The common mistake occurs when creating two new subtasks, rather than only one, which is only needed. The task, which creates the subtasks, can do the work of one of the subtasks. This reduces the overhead massively by eliminating one less thread scheduling [4]. Moreover, a sequential cutoff has to be implemented because parallelizing a task only is advantageous if the overhead of thread creation does not outweigh the actual computation time. Additionally, it has to be considered how data is split between parallel tasks. Copying the data for every thread rather than working on one object or address causes a significant slowdown in computation time. This also applies to merging the results [4].

II. JAVA’S PARALLEL BUILT-IN OPERATIONS

The previous chapter shows that even with the fork-join framework, which solves some of the problems of parallel programming, developing parallel code remains challenging. [5] accurately describes this issue:

Despite the fact that parallel programs are already complex, developers make them even more complex than they need to be.

¹ This abstract is based on a suggestion of key points generated with Gemini Pro 3.1. with prompt “Which key points should an abstract include for this paper?”

Java 8 provides methods for common problems which benefit from parallel execution [6]. Firstly, this leads to a decrease of lines of code since the standard library method handles the majority of the execution. Furthermore, those built-in methods have undergone extensive validation since the release of Java 8 in March 2014 [7]. These factors show the practical advantages of such methods.

A. General structure of the three methods

There are three methods that will be looked at in this paper: `parallelSetAll()`, `parallelPrefix()` and `parallelSort()`. All of those are part of the `Arrays` class as a static method [8], which can be applied to arrays. Each method requires an array of some primitive datatype (e.g. `int[]`, `double[]` etc.) or generic type `T`. An optional range may be specified as an argument to the method with two indexes: `fromIndex` and `toIndex` [8] on which the algorithm will be executed.

B. `parallelSetAll()`

The `parallelSetAll()` method additionally accepts a so called generator as an argument. This generator takes an index and computes a new value for this index [8]. This computation executes for every index in the array (when no range is provided). An example can be seen in Fig. 1 for squaring all numbers in a given array.

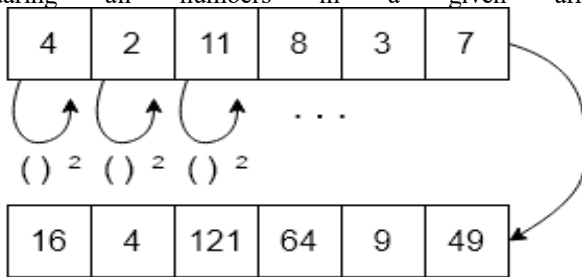


Figure 1. Example of `parallelSetAll()`

Developers can instantiate an `IntUnaryOperator` to pass it to the method which requires overriding the `applyAsInt(int operand)` tailored to the task at hand.

```
1 IntUnaryOperator squaringGen = new
2 IntUnaryOperator() {
3     @Override
4     public int applyAsInt(int operand) {
5         return operand * operand;
6     }
7 };
```

Listing 1. Creating an `IntUnaryOperator` for squaring numbers

An easier way would be to use a lambda expression. This is a short way to input the same information. For this example, the code would be: `Arrays.parallelSetAll(exampleArray, (x) -> x * x);`

```
1 int[] exampleArray = {4, 2, 11, 8, 3, 7};
2 Arrays.parallelSetAll(exampleArray,
3     (x) -> x * x);
3 //result state: {16, 4, 121, 64, 9, 49}
```

Listing 2. Usage of `parallelSetAll`

This way of passing the argument shortens the code by a few lines and improves the readability. This process of inputting is

equivalent for all data types. A more advanced use case for this method would be a Monte Carlo simulation executing several samples from a probability density function concurrently [9].

C. `parallelPrefix()`

Several prefix operations exist. The prefix sum serves as an illustrative example. Prefix sum is calculated by taking the value at index `i` and adding the value of index `i - 1`. The result of it is stored at index `i`. This procedure is depicted in Fig. 2.

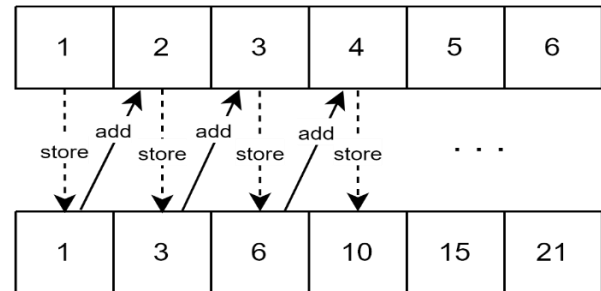


Figure 2 Example of prefix sum.

This method requires a binary operator that accepts two operands and returns a single value. In this case an `IntBinaryOperator` is required. The usage is equivalent to the `IntUnaryOperator` only the signature of the `applyAsInt()` method changes. The method accepts two parameters: `left` and `right` [8]. A shorter and more readable version can be achieved by using a lambda expression: `(left, right) -> left + right`. A common application is computing the cumulative frequency of a given probability distribution.

D. Thread safety

Since all these computations execute in parallel, it is necessary to mention thread safety. For simple calculations, the binary or unary operators are thread safe. To ensure thread safety the lambda cannot use any shared or global variables which are manipulated during the computation. Specifically, a “side effect free” method (short: SEF method) is necessary [8]. A side effect occurs when a method manipulates its state and which can be seen by the code that calls this method [10]. Therefore, the state before and after the method invocation must remain the same. Temporary objects in this method do not affect this state. By default the `parallelSort()` is thread safe but only when no comparator for sorting is used [8]. If a comparator is used, it has to be implemented stateless or free of side effects to ensure thread safety.

E. `parallelSort()`

The `parallelSort()` method requires no unary or binary operator. Implementing a comparator for object types is optional. For primitive datatypes Java uses a Dual Pivot Quicksort developed by Yaroslavskiy [11]. This sorting algorithm also outperforms the classical Single Pivot Quicksort [1]. But for an object type or generics a Timsort is used internally. The Timsort algorithm is a combination of Merge- and Insertionsort that uses the advantages of both [12]. Consequently this sorting algorithm is stable since both

Merge- and Insertionsort are stable [12], [13]. A stable sorting algorithm is essential to preserve the prior sorting order. Quicksort, by contrast, does not preserve pre-existing order, which is irrelevant when sorting primitive data types [13].

III. BENCHMARK OF SORTING METHODS

Since Quicksort is employed, the problems heavy weight splitting and merging, mentioned in chapter I.B, no longer apply. However, Timsort is not in place and needs in the worst-case $O(n)$ extra space for merging [12]. But for this benchmark primitive datatypes will be observed. Therefore the question remains, does the `parallelSort()` method perform well on small arrays and does it have a sequential cutoff? If so, can this threshold be seen in practical measurements?

A. Testing environment

Testing Java code presents several challenges. A primary concern is the garbage collection in Java. This may lead to inference if it takes place during benchmarking [1]. Furthermore, it has to be ensured that the least amount of user programs are running during benchmarking [1]. Additionally, Just-in-time compilation may lead to interference especially during the start of the application [14]. To improve quality and precision of the benchmark the Java Microbenchmark Harness (JMH) was the best choice [15], given it was optimized for small benchmarks. JMH works with annotations to parameterize and customize the benchmark. Moreover, JMH addresses the problems mentioned before. All of the dependencies regarding JMH and the annotation processor are defined in the projects `pom.xml`. The benchmark was conducted on a Galaxy Book2 Pro 360 with Windows 11 operating system, a 12th Gen Intel® Core™ i7-1260P (2.10GHz) processor and 15.6GB of RAM. The available amount of threads for the JVM was 16 and the Heap size was about 4000MB. To access this information the `main` method of the file `HeapInfo.java` can be executed.

B. Benchmark design and implementation

The following listings are part of `ArraySortBenchmark.java`. To test the performance of both methods, integer arrays with various sizes were being sorted. The sizes were: 10, 1,000, 10,000, 100,000 and 10,000,000. These arrays were filled with random numbers. To ensure reproducibility a constant seed was used for the random number generation:

`RANDOM_SEED = 42`. The following setup code was executed before every iteration.

```
1 @Setup(Level.Invocation)
2 public void setup() {
3     sortingArray =
4         new Random(RANDOM_SEED)
5             .ints(size)
6             .toArray();
7 }
```

Listing 3. Setup for sorting Benchmark

For every execution the average time was measured across five iterations. Prior to each benchmark, a warmup was conducted to mitigate the problems mentioned in Section A. This is achieved by adding an annotation above the benchmarking class `@Warmup(iterations = 3,`

`time = 1)`. The code for the benchmark is relatively simple as standard library functions are being tested.

The code for the sequential method is analogous.

```
1 @Benchmark
2 public void benchmarkParallelSort() {
3     Arrays.parallelSort(sortingArray);
4 }
```

Listing 4. Start of benchmark for parallelSort

IV. PERFORMANCE EVALUATION

A. Sorting performance analysis

The following measurements originate from the console output of the JMH. The newest results are saved in `ArraySortBenchmarkResults.json` and can be viewed with the webtool JMH visualizer as diagrams [16]. Each measurement is performed with a confidence interval of 99,9% with the assumption that the runtime of the methods is distributed normally. The whole benchmark took 01:32 minutes.

Table 1. Results of the benchmark

Size	<code>sort()</code> [ms]	<code>parallelSort()</code> [ms]
10 ¹	≈ 0.001	≈ 0.001
10 ³	0.010	0.010
10 ⁴	0.384	0.234
10 ⁶	5.211	1.102
10 ⁷	722.219	108.264

Table 1 reveals two notable observations: First, for larger arrays `parallelSort()` outperforms the sequential method by about 6 seconds at the largest size as expected, given due to the parallelism used. This further indicates that parallel algorithms are faster as discussed in chapter 0. Second, for smaller array lengths the parallel method maintains comparable performance, even with the overhead of thread creation or handling. This suggest that the Java method has a sequential cutoff. The rounded numbers for small arrays are due to the accuracy of the framework.

B. Empirical validation of thresholds

To address the second question another benchmark (`GranularityBenchmark.java`) was implemented to determine if the existence of the sequential cutoff is noticeable in the measurements. Arrays with random generated integers with a seed were sorted with several changes to the setup. The iterations of the measurements were increased to 10, to further enhance precision. Additionally, the sizes of the array were modified to the length 2000, 3000, 4090, 4100, 5000 and 6000. These values were chosen to be

closely around the sequential cutoff of primitive datatypes.

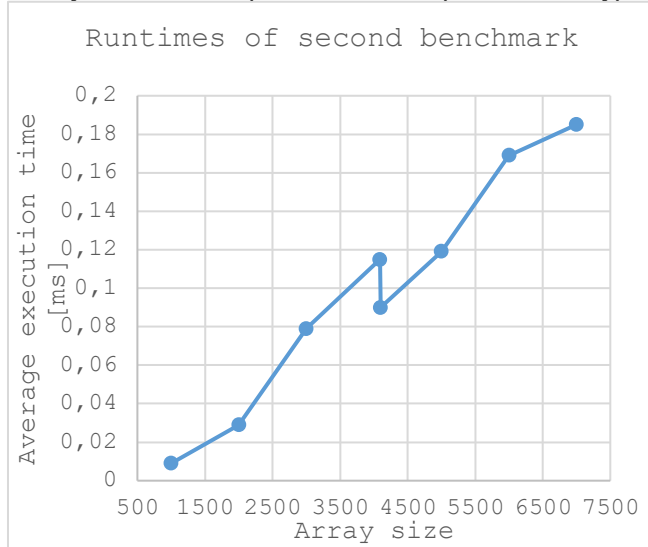


Figure 3. Graph of measurement times

Fig. 3 clearly depicts a decrease in execution time at around 4100 elements. Before and after that the curve is approximately linear. This suggests that the sequential cutoff causes the observed drop.

C. Granularity limits in the Java standard library

A sequential cutoff exists. Examining the Java source code will lead to two specific variables. The `MIN_ARRAY_SORT_GRAN` specifies the boundary for the granularity of a sorting task in the `Arrays` class [6]. If the array length exceeds this constant a parallel algorithm will be used, if not it will be sorted sequentially. This constant has the value `1 << 13` or 8192. But why exactly 8192? [1] explains, the reason for this specific value is the common size of a L1 cache during the time of the release of Java 8 in 2014 [7]. As it provides no benefit to parallelize across multiple cores when all data fits within the cache of one processor. Splitting the task would cause several context switches and sharing data between cores. This constant is only used for generic data types. For primitive datatypes the `sort()` method of the `DualPivotQuicksort` class is invoked. In this class a different threshold is used: `MIN_PARALLEL_SORT_SIZE` with the value `4 << 10` or 4096 [6]. Two reasons explain this difference. Firstly, generics or objects have to be sorted with Timsort to guarantee stability. But this leads to a space complexity of $O(n)$ [12]. This allocation of new space and the garbage collection afterwards results in an increase of execution time. Secondly, the comparison of two object types is more time consuming since that only the references are stored in RAM and the CPU has to load several values. Consequently the threshold is lower for primitive types because parallelizing is already efficient at a lower size. Furthermore, there is even another limit when the array becomes smaller than 65. The sequential Dual Pivot Quicksort will switch to a Insertion Sort [6]. These sequential cutoff mechanisms are also implemented for the `parallelSetAll()` and `parallelPrefix()` method [6].

V. CONCLUSION AND FUTURE WORK

This paper evaluated the performance differences between sequential and parallel array operations in Java using the JMH framework. There are several benefits of using `parallelSetAll()`, `parallelPrefix()` and `parallelSort()` in this paper:

- These methods reduce the lines of code required
- They provide parallelism benefits without the risks of race conditions and deadlocks
- No external libraries are required; every method is part of the `Arrays` class.
- They are applicable in older Java projects, as these methods were released in Java 8, which has long term support [7]

To further validate the measured data the benchmark could be executed on different machines or more isolated environments to reduce interference. Moreover, the benchmarks could be expanded by comparing the sorting algorithms while sorting complex datatypes in contrast to only integers and comparing the runtimes around the sizes of the sequential cutoff. Replicating these benchmarks with other with other distributions beyond random numbers may yield more precise and valuable data. Additionally, altering the sequential cutoff may provide more insights on the overhead of thread creation, potential improvement of performance or linearization of the runtime curve.

ACKNOWLEDGMENT

The author would like to thank Prof. Dr. Tobias Lauer for their support and feedback throughout this project, as well as Nico Disch for proofreading this paper.

REFERENCES

- [1] A. Brooks, L. Krebs, and B. Paulsen, 'A Comparison of Sorting Times between Java 8 and Parallel Colt: an Exploratory Experiment', in *ACM SIGSOFT Software Engineering Notes Volume 41, Issue 4*, 4th edn, vol. 41, in *ACM SIGSOFT Software Engineering Notes*, vol. 41., New York, NY, United States: Association for Computing Machinery, 2016. doi: 10.1145/2967307.
- [2] P. Bertels and D. Stroobandt, 'Java and the Power of Multi-Core Processing', in *2008 International Conference on Complex, Intelligent and Software Intensive Systems*, IEEE, Mar. 2008.
- [3] N. Disch, *Paper_ParallelArrayOperationsJava*. Java. [Online]. Available: https://github.com/nods21/Paper_ParallelArrayOperationsJava
- [4] M. De Wael, S. Marr, and T. Van Cutsem, 'Fork/join parallelism in the wild: documenting patterns and anti-patterns in Java programs using the fork/join framework', in *ACM Conferences PPPJ '14: Proceedings of the 2014 International Conference on Principles and Practices of Programming on the Java platform: Virtual machines, Languages, and Tools*, New York, NY, United States: Association for Computing Machinery, Sep. 2014, p. 214. doi: 10.1145/2647508.

- [5] S. Okur and D. Dig, ‘How do developers use parallel libraries?’, in *FSE ’12: Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*, New York, NY, United States: Association for Computing Machinery, Nov. 2012, p. 494. doi: 10.1145/2393596.
- [6] Oracle Corporation, *Java Development Kit (JDK) 25*. (Jan. 20, 2026). Java. Oracle Corporation. Accessed: Apr. 11, 2026. [Online]. Available: <https://www.oracle.com/java/technologies/downloads/>
- [7] Oracle Corporation, ‘Oracle Java SE Product Releases’, Oracle Java SE Support Roadmap. Accessed: Apr. 20, 2026. [Online]. Available: <https://www.oracle.com/java/technologies/java-se-support-roadmap.html>
- [8] Oracle Corporation, ‘Class Arrays Documentation’, Arrays (Java Platform SE 8). Accessed: Apr. 20, 2026. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html>
- [9] R. L. Harrison, ‘Introduction To Monte Carlo Simulation’, in *AIP Conf Proc. 2010 Jan 5*, Jan. 2010, pp. 17–21. doi: 10.1063/1.3295638.
- [10] A. Rountev, ‘Precise identification of side-effect-free methods in Java’, in *20th IEEE International Conference on Software Maintenance, 2004. Proceedings*, New York, NY, United States: IEEE, Sep. 2004. doi: 10.1109/ICSM.2004.1357793.
- [11] V. Yaroslavskiy, ‘Dual-Pivot Quicksort algorithm’. Sep. 11, 2009. Accessed: May 24, 2026. [Online]. Available: <https://codeblab.com/wp-content/uploads/2009/09/DualPivotQuicksort.pdf>
- [12] K. Bhagat, A. K. Das, S. K. Agrahari, S. A. Shah, D. R. T, and G. Ramasamy, ‘Cross-Language Comparative Study and Performance Benchmarking of Sorting Algorithms’, in *Proceedings of the 3rd International Conference on Optimization Techniques in the Field of Engineering*, SSRN, Oct. 2024. doi: <https://dx.doi.org/10.2139/ssrn.5088751>.
- [13] Z. Inayat, R. Sajjad, M. Anam, A. Younas, and M. Hussain, ‘Analysis of Comparison-Based Sorting Algorithms’, in *2021 International Conference on Innovative Computing (ICIC)*, New York, NY, United States: IEEE, 10.11 2021. doi: 10.1109/ICIC53490.2021.9692969.
- [14] A. Georges, D. Buytaert, and L. Eeckhout, ‘Statistically Rigorous Java Performance Evaluation’, in *ACM SIGPLAN Notices*, New York, NY, United States: Association for Computing Machinery, Oct. 2007, p. 686. doi: 10.1145/1297105.
- [15] OpenJDK, *Java Microbenchmarking Harness (JMH)*. (Mar. 30, 2026). Java. Accessed: May 13, 2026. [Online]. Available: <https://github.com/openjdk/jmh/tree/996afd57d025544adef727ca32e09d18017a72e>
- [16] OpenJDK, *Java Microbenchmarking Harness Visualizer*. [Online]. Available: <https://jmh.morethan.io/>